

EXPLICACION

```
# -----  
class Infija_a_Posfija:  
    def __init__(self):  
        self.stack = Stack()  
        self.output = []
```

En esta parte del código lo primero que hacemos es crear una clase e inicializar todos los elementos que se ocuparan más adelante como lo son la clase de stack que es la de la pila la cual usamos para almacenar los datos en orden y luego se inicializa una lista que será en la que se guarden los datos acomodados de la forma infija.

```
def obtener_precedente(self, op):  
    if op in ('+', '-'):   
        return 1  
    if op in ('*', '/'):   
        return 2  
    if op == '^': # potencia  
        return 3  
    return 0
```

En esta parte lo que se hace es darle la prioridad en la que se acomodara cada signo cómo se muestra el “+” y “-” tienen menor prioridad por lo tanto se le da el valor de 1 y la potencia tiene 3 ya que es el que mayor prioridad tiene.

```
def definir_operador(self, c):  
    return c in ['+', '-', '*', '/', '^']
```

Esta función se utiliza para identificar si un carácter es un operador o no, en caso de que lo sea lo apila en caso de que no lo sea lo manda directo a la lista “output”.

```

def converscion(self, expression: str) -> str:
    self.output = [] # limpiar salida en cada conversión
    for token in expression:
        if token.isalnum(): # operando
            self.output.append(token)
        elif token == '(':
            self.stack.push(token)
        elif token == ')':
            while not self.stack.is_empty() and self.stack.top_of_stack() != '(':
                self.output.append(self.stack.pop())
            self.stack.pop() # eliminar '('
        elif self.definir_operador(token):
            while (not self.stack.is_empty() and
                  self.obtener_precedente(self.stack.top_of_stack()) >= self.obtener_precedente(token)):
                self.output.append(self.stack.pop())
            self.stack.push(token)

    while not self.stack.is_empty():
        self.output.append(self.stack.pop())

    return "".join(self.output)

```

Esta función es la que hace la conversión lo que hace primero es limpiar la lista para en caso de que ya se allá realizado algún problema no de un resultado anterior o no los de mezclados, luego recorre la expresión carácter por carácter luego si el carácter es una letra o un numero los manda directo a la salida luego hace la excepción de los paréntesis en la cual si es "(" los va apilando luego con ")" los desapila hasta encontrarse con el paréntesis de apertura y manda todo lo que estaba apilado a la salida luego se usa la función "definir_operador" para comparar la precedencia de los operadores luego saca todo lo que quedo en la pila al final cuando ya no tiene ningún operador para comparar y devuelve la expresión ya acomodada en posfija.